Headline Generation: LSTM Seq2Seq vs. LLM

A Comparative System Report

Final Project Report

CP468-D - Artificial Intelligence

Wilfrid Laurier University

Link to Github Repo

Watch Video Demo

Group 7

Manahil Bashir

Safdar Md Abdus Shukur

Morad Mohammad

Noah Samarita

Farhan Chowdhury

(See Appendix B for Contribution)

1. Introduction

Headline generation asks a system to condense a full news article into a single, factual line that a reader can scan in place of the article. The task compresses a long, entity-dense input into a short, fluent, and information-preserving output, which makes it a useful stress test for sequence-to-sequence learning: the model must select the right facts, discard the rest, and stay grammatical while doing so.

This project builds a classical LSTM-based encoder-decoder with attention for headline generation, trained entirely from scratch on a small, publicly available news dataset. Its output is compared against a general-purpose large language model (Gemini) prompted in zero-shot and few-shot settings on the identical test set. The goal is not to close the gap between the two systems but to measure it precisely, explain where it comes from, and reason about when a small trained-from-scratch model remains the right engineering choice despite the gap.

2. System Design

2.1 Dataset and Preprocessing

The project uses the Kaggle “News Summary” dataset (uploader: sunnysai12345), a collection of Indian and UK news articles paired with human-written headlines, drawn mainly from India Today, Hindustan Times, and The Guardian between April and May 2017. The dataset is publicly downloadable and cited above; its listed license is GPL-2.0 per the Kaggle publisher, which is a limitation worth stating plainly: GPL-2.0 was written for software, not data, and its applicability to a dataset of this kind is ambiguous. This is flagged directly in the repository's data documentation rather than treated as settled.

Of 4,514 raw rows, 4,341 remained after removing empty rows and duplicate articles, and 2,691 remained after filtering to articles of 20–400 tokens and headlines of 2–30 tokens. The retained data was split 2,152 / 269 / 270 for training, validation, and testing (roughly 80/10/10) using a fixed random seed of 42. To prevent leakage, the split was performed before vocabulary construction, and the source and target vocabularies were built from the training split only.

Preprocessing (src/preprocess.py) proceeds in a fixed order: the raw CSV is read with a UTF-8 → latin-1 → cp1252 fallback chain, article and headline columns are renamed to source and target, empty rows are dropped, text is whitespace-collapsed, stripped, NFKC-normalized, and lowercased, duplicate articles are removed, and text is tokenized with a \w+|[^\w\s] regex. Source and target vocabularies are then built from the training split only, with a minimum frequency of 2 and a maximum size of 30,000 tokens, and sequences are encoded with <bos>/<eos> markers. The resulting vocabularies are small (15,881 source tokens and 2,527 target tokens), which becomes directly relevant to the LSTM's failure modes discussed in Section 4. Variable-length batches are handled with PyTorch's pack_padded_sequence/pad_packed_sequence in the encoder, and a boolean padding mask (source_ids ≠ pad_id) is applied as $-\infty$ to attention scores before the softmax so the decoder cannot attend to padding.

2.2 LSTM Sequence-to-Sequence Model

The model is implemented directly in PyTorch using only standard layers (nn.LSTM, nn.Embedding, nn.Linear) with no prebuilt sequence-to-sequence trainer, per the project constraints. Its architecture is as follows:

- Embedding dimension: 128, shared learned embeddings for source and target vocabularies.
- Encoder: a single-layer bidirectional LSTM with hidden size 256 per direction (512-dimensional output after concatenation), run over packed sequences.
- Bridge: the encoder's final bidirectional hidden and cell states are projected into the decoder's unidirectional 256-dimensional space: the hidden state through a linear layer followed by tanh and the cell state through a linear layer with no activation.
- Attention: Bahdanau-style additive attention, score = $v^T \cdot \tanh(W_1 \cdot h_i + W_2 \cdot s_t)$, with padding positions masked to $-\infty$ before the softmax so attention mass cannot land on padding.
- Decoder: a single-layer unidirectional LSTM whose input at each step is the target embedding (128-d) concatenated with the attention context vector (512-d), producing a 256-dimensional hidden state.
- Output projection: a linear layer mapping the concatenation of the decoder hidden state and the attention context (256 + 512 = 768 dimensions) to the 2,527-word target vocabulary.
- Decoding: teacher forcing during training, greedy (argmax) decoding at inference.

The full model has 6,469,599 trainable parameters. An ablation that removes the attention mechanism reduces this by 197,376 parameters to 6,272,223; this ablation's results are reported in Section 3.

Training used a batch size of 32, the Adam optimizer at a learning rate of 0.001, gradient clipping at a maximum norm of 1.0, and a dropout rate of 0.3, with all seeds (Python, NumPy, and PyTorch) fixed at 42. The loss is cross-entropy with padding tokens ignored; critically, the <unk> token's loss weight was set to 0.0, because <unk> accounted for 13.91% of all training-set headline tokens and would otherwise dominate the gradient signal and be predicted disproportionately often (this is verified directly by the ablation in Section 3, which restores unk_loss_weight to 1.0 and shows the model collapse as a result). Teacher forcing began at a ratio of 1.0 and decayed by 0.02 per epoch. Training was configured for up to 30 epochs with early stopping at a patience of 5; in practice, training stopped after 12 epochs, with the best validation loss of 5.0501 reached at epoch 7. Total training time was 31.5 minutes (roughly 157 seconds per epoch) on an Apple M4 CPU; no GPU was used.

Reproducibility: the repository includes a fully pinned requirements.txt (PyTorch 2.8.0, NumPy 2.0.2, sacrebleu 2.6.0, rouge-score 0.1.2, plus transitive dependencies, verified on Python 3.9), a README with the exact commands to reproduce every result in this report, and fixed random seeds (42) applied consistently across preprocessing, training, and the LLM baseline script. Reproduction requires cloning the repository, installing the pinned environment, running preprocess.py, then train.py with the flags recorded in results/training_config.json, then llm_baseline.py, evaluate.py, and compare_results.py in sequence.

2.3 LLM Baseline

The LLM baseline calls Gemini (model name gemini-3.5-flash-lite, as hardcoded in llm_baseline.py) directly via the Gemini generateContent HTTP endpoint, without an SDK. This model name falls after this report's authors' reliable knowledge horizon and is taken at face value from the repository rather than independently verified.

Two prompt variants were tested on the identical 270-example test set, both at temperature 0.2 with a 40-token output cap. Both are reproduced verbatim below, exactly as defined in llm_baseline.py, with {article} marking where the source article text is inserted.

Zero-shot prompt:

```
You are a professional news editor.

Generate one concise and factual headline for the article below.

Rules:
- Output only the headline.
- Do not add quotation marks.
- Do not invent information.
- Keep the headline under 15 words.

Article:
{article}

Headline:
```

Few-shot prompt (k = 3):

```
You are a professional news editor.

Generate one concise and factual headline for the article below.

Rules:
- Output only the headline.
- Do not add quotation marks.
- Do not invent information.
- Keep the headline under 15 words.

Example 1:
Article:
Apple announced a new series of laptops featuring faster processors and longer battery
life.

Headline:
Apple Unveils Faster Laptops With Longer Battery Life
```

```
Example 2:
Article:
Heavy rainfall caused flooding across several neighbourhoods and forced officials to
close roads.

Headline:
Heavy Rain Triggers Flooding and Road Closures

Example 3:
Article:
The national football team defeated its rival 2-1 after scoring in the final minutes.

Headline:
Late Goal Secures 2-1 Victory for National Team

Now generate a headline for this article.

Article:
{article}

Headline:
```

Running both settings across all 270 test examples (540 requests total) is estimated, using the repository's cost script, at approximately $0.095 total (roughly $0.0002 per request), based on a heuristic of 4 characters per token rather than Gemini's actual tokenizer, and the repository explicitly caveats this as an order-of-magnitude estimate, not an exact figure.

3. Experimental Results and Evaluation

3.1 Automatic Metrics

Two standard metrics were computed on the 270-example test set: corpus-level BLEU (via sacrebleu, case-insensitive) and ROUGE-1 / ROUGE-2 / ROUGE-L F-measure (via rouge-score, with stemming, averaged per example). Lowercasing was applied deliberately for BLEU because the LSTM's vocabulary and output are entirely lowercase by construction, while the reference headlines and LLM outputs retain original casing; comparing case-sensitively would penalize the LSTM for a preprocessing decision rather than a generation error.

System	BLEU	ROUGE-1	ROUGE-L
LSTM + attention	0.4372	0.0821	0.0781
Gemini, zero-shot	10.4586	0.4536	0.3870
Gemini, few-shot (k=3)	10.6955	0.4388	0.3758

(ROUGE-2 was also computed: 0.0117 for the LSTM, 0.2050 for zero-shot, and 0.1944 for few-shot; it is omitted from the table above only for space.)

The gap holds across article length. Splitting the test set into three equal-count-length terciles and comparing ROUGE-1 gives 0.0851 (LSTM) vs. 0.4899/0.4736 (zero-/few-shot) for the shortest third (19–154 tokens), 0.0846 vs. 0.4318/0.4176 for the middle third (154–251 tokens), and 0.0767 vs. 0.4392/0.4252 for the longest third (252–347 tokens): a roughly 5.7–5.8× gap in every bucket. The gap is a consistent property of the model comparison, not an artifact of longer or shorter articles.

3.2 Ablations

- **Unmasking <unk> in the loss (unk_loss_weight = 1.0):** BLEU falls to 0.00, and ROUGE-1/L fall to 0.0067, and the decoder collapses into predicting <unk> disproportionately, confirming that the 0.0 weighting used in the main model is necessary, not cosmetic.
- **Free-running validation loss:** validation computed without teacher forcing was tracked as a training-curve diagnostic only; it did not produce a scored checkpoint, and early stopping under this criterion selected epoch 1, which emitted an identical headline for every article, a useful illustration of how fragile early-stage seq2seq training is before attention has learned to differentiate inputs.

- **No-attention ablation:** this comparison (bidirectional encoder without attention vs. with attention) was not run before this report was written. It is a genuine gap and is carried into Section 4.6 as a limitation rather than assumed.

3.3 Qualitative Comparison and Error Analysis

The repository's automatic error-analysis pass (results/error_analysis.md) reports proxy heuristics rather than the assignment's manual PRD categories directly: 41.5% of LSTM outputs fall under a repetition threshold (fewer than 70% unique bigrams), 24.4% exceed the reference/prompt length expectations, and 84.8% are flagged by an "unsupported content" proxy (content in the output not traceable to the source). For Gemini, the unsupported-content proxy is 4.1% (zero-shot) and 5.2% (few-shot), with 1.1% and 0% prompt-format violations, respectively. Separately, 23.56% of all reference-headline tokens fall outside the LSTM's 2,527-word target vocabulary, and this OOV ceiling touches 95.6% of references, meaning almost every reference headline contains at least one word the LSTM is structurally incapable of producing.

Manually reviewing ten test examples against the assignment's PRD failure categories (under-translation, repetition, hallucination, OOV/vocabulary failure, and fluency vs. adequacy) confirms and sharpens this picture rather than just restating the proxy numbers. Three representative rows are shown below; the complete 10-example side-by-side comparison, with source-article topic, reference, both system outputs, and a PRD category tag for every example, is provided in Appendix A.

#	Source (topic)	Reference	LSTM output	Gemini output (few-shot)	PRD category
1	Celebrity brand-ambassador announcement for a luggage company	American Tourister appoints Kohli as brand ambassador	"india's son's son's son's son"	American Tourister Appoints Virat Kohli as Brand Ambassador	LSTM: repetition; under-translation (all source content dropped). Gemini: none observed.
6	Transit-authority announcement of a new mobile ticket-booking method	Mumbai railways to allow ticket bookings on feature phones	"china, 000 kg for smuggling - old boy"	Railways to Introduce USSD Code System for Mobile Train Ticket Booking	LSTM: hallucination (unrelated topic); OOV failure ("USSD"/domain terms fall outside the 2,527-word vocabulary). Gemini: none observed.
10	Public controversy over a celebrity's remarks in a criminal case	Kamal Haasan criticised for naming actress in abduction case	"i's son's son's son's son: report"	Kamal Haasan Criticized for Revealing Name of Assault Survivor	LSTM: repetition; under-translation. Gemini: fluency vs. adequacy ("abduction" → "assault" shifts meaning).

Two patterns hold consistently across all ten examples (Appendix A). First, the LSTM's dominant failure mode is repetition: looping on a possessive fragment ("'s son's son…") or a single word ("…old-old-old…") rather than producing a complete, novel sentence. This co-occurs with under-translation and, in the more extreme cases, hallucination: the output frequently has no traceable relationship to the source article at all (examples 1, 6, 9, 11), which is what the 84.8% unsupported-content proxy is capturing rather than overstating. Several of these breakdowns trace directly to OOV failure: domain-specific or proper-noun tokens ("USSD," "Poonch," "J&K") fall outside the 2,527-word target vocabulary, so the decoder falls back on high-frequency filler it can generate, consistent with the 23.56% OOV ceiling. Second, Gemini's failures are of a different character entirely and are much rarer: near-perfect format compliance (0% violations in few-shot), occasional over-elaboration or insertion of a plausible-but-unverified specific detail not confirmable from the reference alone (examples 5, 11), and occasional fluency-vs.-adequacy drift where a semantically close but not identical word is substituted (example 10). These are the categories the assignment brief anticipates for LLM baselines, and they are borne out directly in this data rather than assumed from general knowledge of LLM behaviour.

4. Discussion

4.1 Quantitative Gap

The gap is large and metric-dependent. BLEU shows roughly a 24× gap (0.44 vs. 10.46–10.70), while ROUGE-1/L show a roughly 5–6× gap. This is informative: BLEU's brevity penalty and strict n-gram precision punish the LSTM's repetition-loop outputs far more harshly than ROUGE's recall-weighted overlap, since a repeated word loop contributes almost no unique n-gram matches to BLEU but still registers partial overlap under ROUGE. The metrics agree on which system is better; they disagree only on the size of the gap, a reminder that metric choice shapes the story a report tells.

4.2 Why the Gap Exists

Three factors from the course map are at play in this result. Model capacity and data scale: 6.47M parameters trained on 2,152 examples versus a model pretrained on web-scale corpora orders of magnitude larger, giving Gemini a general command of grammar, entities, and world knowledge the LSTM cannot acquire from 2,152 headlines. Transfer learning: Gemini's headline-writing ability is a downstream application of pretraining, not something learned from this dataset, whereas the LSTM's only source of language knowledge is this one small corpus. Architecture: even with attention giving the decoder access to all encoder states, a 1-layer, 256-unit recurrent model has far less capacity per parameter than a transformer, and the practical bottleneck is the 2,527-word target vocabulary forced by the small training set; the model is architecturally attentive but lexically starved.

4.3 Failure Mode Contrast

The contrast documented in Section 3.3 matches the pattern the assignment brief predicts, and this project's data confirms it directly rather than by assertion: the LSTM fails by looping and losing the topic entirely, driven by its narrow vocabulary and small training set; Gemini fails, when it fails at all, by being slightly too confident: adding a specific detail that cannot be verified from the reference or reframing a concept in adjacent-but-not-identical language. The LSTM's failures are catastrophic and frequent; Gemini's are subtle and rare.

4.4 Fairness of the Comparison

The comparison favors Gemini beyond raw parameter count. Gemini was very likely pretrained on text overlapping this dataset's source publications (India Today, Hindustan Times, The Guardian) and possibly the dataset itself, since it is a public, several-year-old Kaggle dataset likely scraped into web-crawl training corpora. This is a live data-contamination risk that may inflate Gemini's scores relative to genuine zero-shot generalization. The LSTM, in contrast, saw only 2,152 training examples and nothing else. A fairer middle-ground comparison, not run here, would fine-tune a small pretrained transformer (e.g., distilled BART or T5-small) on the same 2,152 examples, isolating pretraining and architecture from raw scale.

4.5 Engineering Trade-offs

Under realistic constraints, the two systems suit different situations. The LSTM has near-zero marginal cost and no network dependency once trained, runs inference in a single local CPU forward pass, and keeps article text on-device, relevant for cost-sensitive, offline, or privacy-sensitive deployments. Gemini costs an estimated $0.0002 per request, depends on network access, and sends article text to a third party. The LSTM's failure surface is also more predictable, confined to its known 2,527-word vocabulary, whereas an LLM's larger vocabulary makes rare failures harder to anticipate. In 2026, a small trained-from-scratch model would still be right for a narrow, high-volume, offline, or low-resource-language setting where an API dependency is unacceptable, but at this project's data scale (2,152 examples) the LSTM has not reached a quality level where that trade-off is attractive.

4.6 Limitations and Ethics

- **Metric limitations:** BLEU and ROUGE reward n-gram overlap and can underscore a legitimate paraphrase (e.g., Gemini's “assault survivor” for the reference's “actress” in example 10).
- **Dataset licensing:** the dataset's GPL-2.0 designation is a software license applied to a data artifact; its scope for this use case is unresolved.
- **Data contamination:** Gemini's pretraining corpus plausibly includes this dataset's source articles, inflating its measured performance versus a genuinely unseen test set.
- **Incomplete ablation coverage and compute:** the no-attention ablation was not run, so attention's contribution cannot be isolated here (the loss-weighting ablation isolates a vocabulary problem, not an architectural one); separately, the LSTM trained in 31.5 minutes on a laptop CPU at negligible energy cost, while each Gemini call draws on data-centre-scale compute not reflected in the numbers above.

Appendix A: Complete Qualitative Comparison

All ten test examples referenced in Section 3.3, with source-article topic (a brief non-verbatim descriptor; full article text is withheld here both for report length and because the source articles are licensed news content, and is available in the repository's results/qualitative_comparison.md and data/processed/test.jsonl), reference headline, both system outputs, and a PRD category tag for every example.

#	Source (topic)	Reference	LSTM output	Gemini output (few-shot)	PRD category
1	Celebrity brand-ambassador announcement for a luggage company	American Tourister appoints Kohli as brand ambassador	“india's son's son's son's son”	American Tourister Appoints Virat Kohli as Brand Ambassador	LSTM: repetition; under-translation. Gemini: none observed.
2	Court ruling on broadcast of Indian TV content in Pakistan	Pakistan's High Court lifts ban on airing Indian TV serials	“pakistan court to be a in in india: report”	Pakistani Court Lifts Ban on Indian Television Shows	LSTM: repetition; fluency (ungrammatical). Gemini: none observed.
3	Celebrity engagement announcement, South Indian film industry	Malayalam actress Bhavana gets engaged to producer Naveen	“indian's oldest's oldest's son's son”	Malayalam Actor Bhavana Engaged to Producer Naveen	LSTM: repetition; under-translation. Gemini: none observed.
5	Unusual theft reported at a bank branch	Naked thief steals shoes, utensils from bank instead of cash	“man arrested for stopping - old girl in delhi's house”	Thief Steals Guard's Shoes and Utensils From SBI Branch, Leaves Money Untouched	LSTM: hallucination. Gemini: fluency vs. adequacy (unverifiable specific detail, bank name).
6	Transit-authority announcement of a new mobile ticket-booking method	Mumbai railways to allow ticket bookings on feature phones	“china, 000 kg for smuggling - old boy”	Railways to Introduce USSD Code System for Mobile Train Ticket Booking	LSTM: hallucination; OOV failure (domain terms outside vocabulary). Gemini: none observed.
9	Former intelligence official's statement on a high-profile capture	Ex-ISI official agrees with India on Jadhav's capture	“sc-yr-old-old-old-old…” (“old” × 16)	Ex-ISI Official Admits Kulbhushan Jadhav Was Captured in Iran	LSTM: repetition (severe). Gemini: none observed.
10	Public controversy over a celebrity's remarks in a criminal case	Kamal Haasan criticised for naming actress in abduction case	“i's son's son's son's son: report”	Kamal Haasan Criticized for Revealing Name of Assault Survivor	LSTM: repetition; under-translation. Gemini: fluency vs. adequacy (meaning shift).
11	Government minister's warning on a militant group's threat	ISIS wants to carry out chemical attacks on UK: Minister	“india's oldest to to be at india's resignation”	Islamic State Seeks Mass Casualty Attack and Lacks Moral Barrier to Chemical Weapons	LSTM: repetition; hallucination (topic drift). Gemini: hallucination (interpretive over-elaboration).
12	Border-region military incident with civilian casualties	Army jawan, 8-year-old girl killed in firing by Pakistan	“indian-yr-old girl to in in in in in j&k”	Soldier and Girl Killed in Pakistani Shelling Along LoC in Poonch	LSTM: repetition; fluency; OOV failure (place names outside vocabulary). Gemini: fluency vs. adequacy (age detail omitted).

Appendix B: Contribution Statement

- **Manahil Bashir, Data Pipeline & Repository Lead:** cleaned and tokenized the dataset, built the vocabulary, implemented padding/masking, set up the leakage-free train/validation/test split, and set up the GitHub repository and Python environment.
- **Safdar Md Abdus Shukur, Custom LSTM Seq2Seq Engineer:** implemented the encoder-decoder architecture with attention from scratch in PyTorch, trained the model, tuned hyperparameters, and logged training curves.
- **Morad Mohammad, LLM Baseline & Prompting Lead:** set up the Gemini API baseline and tested the zero-shot and few-shot (k=3) prompt variants across the test set.
- **Noah Samarita, Evaluation & Qualitative Error Lead:** ran BLEU/ROUGE automatic metrics across all outputs, selected diverse test examples for side-by-side comparison, and built the automatic error-analysis heuristics.
- **Farhan Chowdhury, Report & Video Producer:** led the write-up of this system report, and recorded and edited the demo video.

Appendix C: AI Disclosure Details

- Kimi K2.7 was used to generate initial boilerplate and file scaffolding for LSTM implementation.
- Claude Opus 5 was used for: Debugging two defects that were found and fixed involving decoder collapse. The failed training runs with these bugs are documented in /results. And to help build and debug the data preprocessing pipeline, fix errors, adapt the code to the Kaggle News Summary dataset, and troubleshoot the Python/GitHub setup
- ChatGPT 5.6 used for troubleshooting as well as interpreting LSTM training , Google Gemini 3.5 Flash Lite for llm baseline for headline generation.
- Claude Sonnet 5: Used for analyzing the GitHub to analyze for any errors and pulling the required information for the report in a more organized draft.

References

Kaggle. “News Summary” dataset (sunnysai12345). https://www.kaggle.com/datasets/sunnysai12345/news-summary

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR 2015.

Post, M. (2018). A Call for Clarity in Reporting BLEU Scores (sacrebleu). https://github.com/mjpost/sacrebleu

Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. rouge-score library. https://github.com/google-research/google-research/tree/master/rouge

PyTorch. https://pytorch.org

Google. Gemini API documentation. https://ai.google.dev

Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach (4th Edition). Pearson, 2020.